

Documentation Technique

Architecture, base de données et guide de démonstration

Projet :	Système de réservation Savora
Épreuve :	BTS SIO – E6 (option SLAM)
Version :	1.0
Date :	Mars 2026
Organisation :	Restaurant gastronomique Savora
Candidat :	Balla Gueye — N° 02149092520

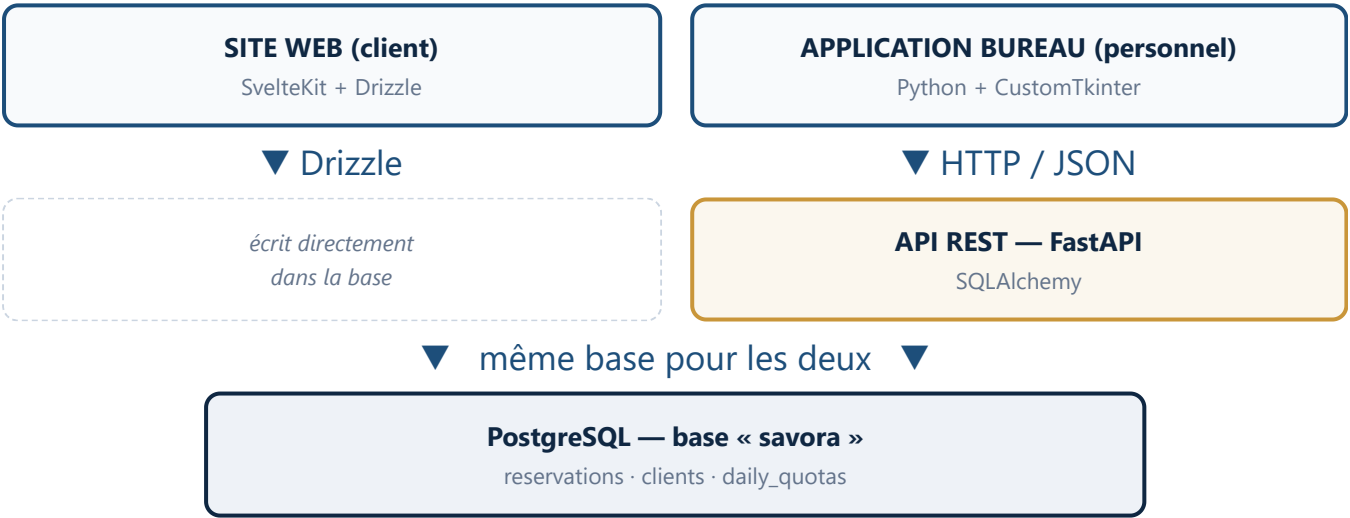
1. Présentation du projet

Savora est un système de réservation pour restaurant composé de **trois parties** qui partagent une **même base de données PostgreSQL** :

Partie	À quoi ça sert	Qui l'utilise
Site web (client léger) SvelteKit + Drizzle	Présenter le restaurant et réserver une table en ligne	Le client
API REST FastAPI + SQLAlchemy	Point d'entrée du client lourd vers la base	(interne)
Application bureau (client lourd) Python + CustomTkinter	Gérer les réservations, les clients et les places	Le personnel

1.1 Schéma de l'architecture

Le **site web** écrit directement dans la base avec **Drizzle**. L'**application bureau** passe par l'**API**. Les deux utilisent la **même base** : une réservation faite sur le site apparaît aussitôt côté personnel.



1.2 Technologies utilisées

Outil	Rôle
PostgreSQL	Base de données (stocke tout)
SvelteKit	Site web (vitrine + réservation)
Drizzle ORM	Accès à la base depuis le site web (en TypeScript)
FastAPI	API REST utilisée par l'application bureau
CustomTkinter	Application bureau du personnel (Python)

2. Mise en route et démonstration

Cette partie explique **comment lancer le projet** et **comment faire une démonstration** complète.

2.1 Lancer le projet

On ouvre **plusieurs terminaux**, dans cet ordre :

① Base de données (PostgreSQL)

```
sudo service postgresql start → base « savora » sur le port 5432
```

② API REST (pour l'application bureau)

```
cd savora_api  
uvicorn main:app --reload → http://localhost:8000/docs
```

③ Site web

```
cd "Client Leger"  
npm run dev → http://localhost:5173
```

④ Application bureau (personnel)

```
cd savora_desktop  
python main.py
```

2.2 Visualiser la base — Drizzle Studio

Drizzle fournit une interface web pour **voir et modifier la base** très simplement. On la lance dans un terminal dédié :

```
cd "Client Leger"  
npm run db:studio → https://local.drizzle.studio
```

Dans le navigateur, on voit les **trois tables** du projet (`reservations` , `clients` , `daily_quotas`). On peut cliquer sur une table pour afficher son contenu, ses colonnes et ses liens.

À montrer au jury : Drizzle Studio est la façon la plus simple de prouver que les données sont bien enregistrées en base après une action.

2.3 Scénario de démonstration (3 étapes)

Étape	Action	Résultat visible
1	Sur le site web , faire une réservation	Message « demande enregistrée »
2	Ouvrir Drizzle Studio et rafraîchir	La réservation et la fiche client apparaissent en base
3	Dans l' application bureau , cliquer  puis « Confirmer »	La réservation passe au statut « confirmée »

Cette démonstration prouve la **cohérence** : le site et l'application bureau travaillent sur la même base.

3. Où réaliser chaque besoin

Ce tableau indique, pour chaque besoin, **où aller dans le projet** pour le réaliser. Il sert de repère pendant la présentation.

Besoin	Où le faire	Comment
Réserver une table en ligne	Site web	Formulaire : couverts, date, horaire, coordonnées
Voir les places disponibles	Site web	Les créneaux complets sont grisés automatiquement
Consulter les réservations	Application bureau	Planning du jour, navigation par date
Filtrer (midi/soir, statut)	Application bureau	Boutons en haut + onglets de statut
Valider une réservation	Application bureau	Bouton « ✓ Confirmer »
Refuser une réservation	Application bureau	Bouton « ✕ Refuser »
Modifier une réservation	Application bureau	Panneau de détail (couverts, note)
Créer une réservation (téléphone)	Application bureau	Bouton « + Nouvelle réservation »
Gérer les clients (créer/modifier/supprimer)	Application bureau	Onglet « 👤 Clients »
Voir l'historique d'un client	Application bureau	Cliquer un client → fiche détaillée
Limiter le nombre de couverts	Application bureau	Barre « Quota » en bas
Vérifier les données en base	Drizzle Studio	<code>npm run db:studio</code>

4. La base de données

Base : savora | **Moteur :** PostgreSQL | **3 tables.** On peut afficher ces tables en direct avec **Drizzle Studio** (voir § 2.2).

4.1 Table « reservations »

Une ligne = une réservation (créée par le site ou saisie par le personnel).

Colonne	Type	Description
id	entier	Identifiant unique
party_size	entier (1–6)	Nombre de couverts
date	date	Jour de la réservation
time_slot	texte	Heure (ex : 19:30)
meal_period	énum	midi (lunch) ou soir (dinner)
civility / first_name / last_name	texte	Identité du client
phone / email	texte	Coordonnées (l'e-mail relie au client)
status	énum	en attente / confirmée / refusée / annulée
staff_note	texte	Note interne du personnel

4.2 Table « clients »

Une ligne = un client. Créée automatiquement à la première réservation (clé : l'e-mail).

Colonne	Type	Description
id	entier	Identifiant unique
civility / first_name / last_name	texte	Identité
phone	texte	Téléphone
email	texte (unique)	Identifie le client de façon unique
notes	texte	Notes internes

4.3 Table « daily_quotas »

Une ligne = la capacité maximale d'un service (midi ou soir) pour un jour donné.

Colonne	Type	Description
id	entier	Identifiant unique
date	date	Jour concerné
meal_period	énum	midi ou soir
max_covers	entier	Nombre de couverts maximum (15 par défaut)

5. Comment ça marche (l'essentiel)

5.1 Le site web et Drizzle

Quand un client réserve, le site enregistre la réservation **directement dans PostgreSQL avec Drizzle**.

Exemple simplifié de l'insertion :

```
// le site insère la réservation dans la base
await db.insert(reservations).values({
  partySize, date, timeSlot, mealPeriod, ... , status: 'pending'
});
```

Le site calcule aussi les **places restantes** (avec Drizzle) pour griser les créneaux complets. *Aucun e-mail n'est envoyé* : la réservation reste « en attente » jusqu'à validation par le personnel.

5.2 L'application bureau (MVC)

L'application du personnel est organisée en **MVC** (3 rôles séparés) :

Rôle	Fait quoi	Fichier
Contrôleur	Coordonne l'écran et les actions	main.py
Vues	Les écrans (planning, détail, clients, quota)	components/
Modèle	Parle à l'API REST	api_client.py

Toutes les actions (valider, refuser, modifier...) passent par l'API, qui applique les règles et enregistre dans la base.

5.3 La règle des places (quota)

Avant d'enregistrer une réservation, on vérifie qu'il **reste de la place**. Si le service est complet, la réservation est refusée.

```
if (couverts_réservés + nouveaux_couverts > maximum) {
  refuser ; // service complet
}
```

Phrase simple à retenir : « Le site écrit dans la base avec Drizzle, l'application bureau passe par l'API, et les deux partagent la même base PostgreSQL. »

6. Sécurité (l'essentiel)

Risque	Protection mise en place
Injection SQL	On n'écrit jamais le SQL « à la main » : Drizzle et SQLAlchemy génèrent des requêtes sûres
Trop de réservations	Vérification du quota : refus si le service est complet
Données invalides	Contrôles avant enregistrement (couverts 1–6, date non passée, e-mail valide)
Accès à la base	Le navigateur n'accède jamais à la base ; seul le serveur le fait
Mots de passe / accès	Rangés dans un fichier <code>.env</code> (hors du dépôt Git)
Doublon client	L'e-mail est unique : pas deux fiches pour la même personne

Si le jury demande « montrez-moi... » : se référer au tableau du § 3 (« Où réaliser chaque besoin ») pour aller directement au bon endroit.